

message-passing mechanism, which is different from traditional parallel computing systems. The main drawback of the commonly-used Message Passing Interface is the lack of elasticity, in addition to extra processors in the middle of computation. Again, this standard suffers greatly in fault recovery and for cloud computing.

The one-sided communication used in Julia resembles higher-level operations like function calls. This looks quite different from Message Sent and Message Received that we see in conventional two-sided communication.

Parallel programming is built on remote references and remote calls. An object stored in a particular process, which can be accessed from any process, is called a remote reference. Request by a process to call a certain function on certain arguments on another process is referred to as a remote call.

**Calling functions from C and Python.** Although Julia offers a good environment for scientific programming, there are already a handful of high-quality libraries in C and Fortran for numerical programming. The functions can be called directly and efficiently without the need for any glue code generation or compilation.

PyCall package helps the user to call Python functions from Julia. This package also allows the user to have automatic-type conversion between Python and Julia.

**Meta-programming support.** Another notable feature of Julia is its meta-programming support. Following Lisp style, the code in Julia is represented as a data structure of

the language itself. Transformation and generation of the code by the program is possible as creation and manipulation of objects could be done within the language itself.

Inclusion of the generated code into the final body of the program is accomplished with the help of macros. Macros in Julia are hygienic in the sense that, expansion of macros happens without causing accidental capture of identifiers.

### What the users feel

Julia was designed to do everything in programming. While serving as a high-level technical language like MATLAB, it is also capable of general-purpose programming as in Python. While the design facilitates parallel and distributed programming as in Hadoop, it does not compromise on speed, which is equivalent to C or Java.

User community across the world is very excited about this software. A user from ResearchGate community feels, "As it looks now, speed is the biggest advantage of Julia. I have been inclined to move to C/C++ for some of my large-scale simulations for time considerations and Julia is giving a viable alternative. It may really be filling a niche there, and I think it could become a big player for heavier computing; unless Python catches up in time."

"It might be a good idea to switch to Julia because when you factor in computation time, the amount of time you took to write and debug your code, Julia may beat Fortran in many situations," points out another user.

But some users are of the opinion

that the software is too young to be judged. "I think we should let time decide. Julia is nice and fast but unstable and improving fast. I personally use Python for most of my computation. I tried programming with Julia a bit and I think it could be a good rival to Python/R/MATLAB/Octave, but it will take time for Julia to get a big community and support like Python/MATLAB," says another user. A user from *stackexchange.com* estimates that it may take about ten years for Julia to become mainstream.

### To be the C for scientists

Being a techie, by now you must have come across many programming languages. Each language will be good at something but not everything. Conventionally, a high-level dynamic programming language might not be that good in performance. There arise issues in distributed computing, regarding copyright, etc. Ultimately, you will end up switching many languages before the end of your project.

Julia is designed to be a unique solution—a single dynamic programming language that does not compromise on productivity or performance. "People have assumed that we need both fast and slow languages," says Bezanon, co-creator of Julia. "I happen to believe that we do not need slow languages."

Can Julia be a high-performing open source MATLAB alternative or a technical rival for C? Time will tell! ●

*The authors are assistant professors in ECE department at SETCEM, Thrissur*

THE COMPLETE MAGAZINE ON OPEN SOURCE

**OpenSource**  
ForYou

Your favourite Magazine on  
Open Source is now on the Web, too.

**OpenSourceForU.com**

Follow us on Twitter@LinuxForYou